Question 1:- Empirical Analysis of Hyperparameters N and k

1.1 Experimental Setup and Optimization Strategy Initially, I concentrated on mapping the k-Nearest Neighbor (k-NN) algorithm's performance by methodically modifying its two most important variables: the number of nearest neighbors (k) and the training samples per class (N). I made use of the Fashion-MNIST dataset, which has 70,000 grayscale photos in ten different categories of clothing. I used a stratified sampling strategy to guarantee a fair test, extracting precisely N images for each category, with N ranging from 1000 to 6000 in increments of 500.

1.2 Global Accuracy Trends and Data SaturationA distinct logarithmic growth pattern was revealed based on the outcomes in the optimized dataset. The global accuracy for k = 5 was roughly 82.56% when N = 1000 samples were used per class. Up until roughly N = 3500, the accuracy quickly improved as I increased the data.

The accuracy peaked at about 85.39% by the time I reached the maximum N = 6000. But after N = 4500, the rate of improvement considerably decreased. This points to a saturation point in the feature space; beyond this point, the model has already captured the main variations of the garment shapes, so adding more samples yields diminishing returns. Rather than elucidating new decision boundaries, additional data primarily serves to reinforce preexisting ones.

1.3 The Impact of the Neighbor Parameter k The choice of k was the main factor in determining whether the model generalized well or became too sensitive to noise.

- **Low k values (k = 1, 2):** These settings were very unstable. Although k = 1 is a common baseline, it is highly vulnerable to "noisy" images, which include situations in which a

garment may have peculiar textures or be at an odd angle. According to my data, in certain trials, k = 1 performed almost a full percentage point worse than k = 5.

- **The Optimal Range (k = 5, 10):** The most consistent values were these ones. The accuracy of k = 10 peaked at 85.36% at N = 5500. The algorithm eliminates individual pixel noise while maintaining enough focus to detect minute differences between objects by using a majority vote among five to ten neighbors.

- **High k values (k = 15, 20):** As the value of k increased towards 20, the accuracy started to decrease. The "voting pool" is too large and begins to accumulate "impostor" points from the visually similar classes.

1.4 Class-Level Stability and Semantic Challenges One of the most interesting parts of this analysis was looking at **local accuracy**. Even with the best hyperparameters, the model struggled with specific categories.

- **Easy Classes:** Class 1 (Trouser) and Class 8 (Bag) were always the top-performing ones, and their accuracy could reach 97.9% and 95.1%, respectively. The reason is that their silhouettes are quite unique and very easy to be recognized by the L2 metric.

- **The "Shirt" Bottleneck:** Class 6 (Shirt) was a major failure point, often struggling to even reach 60% accuracy. The raw pixel intensities of a "Shirt" are mathematically too similar to "T-shirts" (Class 0) and "Coats" (Class 4). Since k-NN doesn't "understand" concepts like buttons or collars, it frequently misclassifies these items, treating them as interchangeable pixel clusters.

1.5 Section Conclusion For this particular case, the identified parameters were that the number of samples, N, should be 6000, and the number of clusters, k, should be 10. Although an 85.36% global accuracy is an excellent starting point, the persistent nature of the confusion in the "Shirt" category indicates that perhaps the L2 metric is too sensitive to changes in the pixel values. This directly relates to the next part of the project, where I shall be investigating whether the metric or sampling process can be improved in these particular trouble spots.

Section 2:- Influence of Indexing Methods on Model Bias

2.1 Comparative Methodology: Random vs. Sequential In the second part of this project, I sought to determine whether the process of data selection, or the order in which the samples were chosen from the original data set, imparted any hidden bias into the k-NN algorithm. Though the standard best practice in machine learning is the use of the Random Indexing method, I chose to use the Sequential Indexing method, where the first N samples from each class were chosen sequentially from the original data file, to determine whether the Fashion-MNIST data set had any inherent ordering or structural organization that might impart bias into the final result.

2.2 Global Accuracy Comparison The empirical results demonstrate an impressive level of consistency between the two indexing approaches. In particular, at the highest training density (N=6000 samples per class), the accuracy gap between Random and Sequential sampling approaches was less than 0.2%. As an example, at k=5 and N=6000, the Random approach yielded 85.39%, while the Sequential approach yielded 85.24%.

2.3 Class-Wise Stability Analysis In order to ensure that there are no biases, I checked the Class Stability line plot. This plot confirms that the performance profile for the ten categories of apparel remains almost identical for both indexing methods. The model continues to perform exceptionally well for Trousers (Class 1) and Bags (Class 8), while it continues to experience a performance plateau for the Shirt category (Class 6).

Since the line graph for the Sequential Indexing method resembles the Random Indexing graph so closely, I can safely conclude that the classification errors are a result of the inherent visual similarity between upper-body garments such as Shirts, Coats, and T-shirts, rather than "luck" with respect to which images are randomly selected.

2.4 Section Conclusion The outcome of this trial clearly demonstrates that the Fashion-MNIST dataset is naturally well-distributed. Regardless of whether the samples are randomly or sequentially selected, the k-NN algorithm clearly converges towards the same levels of accuracy. Therefore, it is now possible to draw the conclusion that the dataset is unbiased with respect to the storage order. However, as is the requirement of scientific research and the standard operating procedures of the industry, I will continue to use the Random Indexing method for the remainder of this project.

Question 3:- Impact of Distance Metrics on Image Classification

3.1 Comparative Methodology: Manhattan (L1) vs. Euclidean (L2) In the final experimental phase, the classification accuracy of the k-NN algorithm is tested based on the mathematical definition of the term "distance." In the preceding sections, the Euclidean distance, or L2 metric,

which is the straight-line distance between two points, is used. In this trial, the Manhattan distance, or L1 metric, is used. In the L1 metric, the distance is defined as the sum of the absolute difference between the pixels.

Both the L1 and L2 metrics were tested with training volumes of 20,000, 40,000, and 60,000 samples with k values of 5 and 10. The performance of the two metrics is visualized in the Metric Comparison Bar Chart.

3.2 Results and Global Performance Analysis The empirical data reveals a consistent advantage for the Manhattan (L1) metric for all test cases. As revealed by the comparative chart, the heights of the L1 columns are greater than the heights of the L2 columns for all categories of N. In particular, for the maximum training volume, i.e., N = 6000, and k = 5, the Manhattan metric resulted in a global accuracy of 86.13%, whereas the Euclidean distance metric resulted in a global accuracy of 85.39%.

The significant difference in the results reveals the relative suitability of the Manhattan distance metric for the Fashion-MNIST dataset. This is because, for clothing, there may be variations in particular regions of the clothing, e.g., a longer sleeve or a different logo location. Thus, the linear approach of the Manhattan distance metric would offer a more reliable measure of similarity between images than the Euclidean distance metric, as the squared Euclidean distance would be significantly affected by minor changes.

3.3 Class-Specific Robustness From the analysis of the performance of the L1 metric on a per-class basis, it is observed that the L1 metric offered significant stability on the "ambiguous"

classes. Though "Shirt" or Class 6 remained the class with the lowest performance for both metrics, as it is more similar to "Coats" and "T-shirt," the Manhattan metric offered a better floor value for the accuracy. In addition, the precision of the "Sandal" and "Pullover" classes is higher with the L1 metric, which validates the effectiveness of the grid-based distance computation.

3.4 Section Conclusion The results obtained in this experiment confirm that the distance metric plays an important role in the performance of the k-NN classifier. In this experiment, the Manhattan (L1) distance performed better than the standard Euclidean (L2) distance by nearly 1% in global accuracy. Based on the results obtained in this experiment, I suggest that the Manhattan distance with a large training set (N = 6000) and k = 5 or 10 should be used for this classification problem.

Question 4:- Final Summary and Synthesis of Findings

4.1 Stability of Local vs. Global Accuracy

Throughout this entire investigation, it has become clear that although increasing the training set (N) and refining the neighbors (k) will improve global accuracy, the local accuracy for each class remains remarkably consistent.

For example, across the entire range of trials, it has been clear that Class 1 (Trouser) and Class 8 (Bag) will always retain an accuracy rate above 95%, whereas Class 6 (Shirt) will never manage to rise above the 55-60% mark, regardless of the Random/Sequential indexing or L1/L2 distance methods. It would seem to suggest that the success or failure of the k-NN algorithm with the

Fashion-MNIST dataset has more to do with the "semantic difficulty" of the images themselves than the actual hyperparameter settings.

4.2 Explaining the "Hard" Classes

The difficulty with the Class 6 ("Shirt") is likely due to the high degree of visual overlap. In the 784-dimensional vector space, the class "Shirt" is mathematically close to the class "T-shirt," "Coat," and "Pullover." In the k-NN classifier, the class is voted on based on the majority of the nearest neighbors. Therefore, the classes overlap. Increasing the amount of data (N) somewhat alleviates the issue but does not address the underlying issue that the classes share similar silhouettes.

4.3 Visual Structure and Algorithm Expectations

The performance of this algorithm was highly related to the "visual structure" of the data. The classes for which k-NN performed best (Trousers and Bags) are classes with highly distinguishable and uniform shapes. A pair of trousers has a long shape with two distinguishable columns of pixels, while a bag usually has a uniform rectangular block shape.

On the other hand, the "Upper Body" clothing class (Classes 0, 2, 4, and 6), which includes items such as shirts, dresses, and skirts, forms a huge cluster with complex shapes in the central region of the data distribution. The use of the Manhattan (L1) distance metric for Question 3 was the best decision because it helped to reduce the effect of the "curse of dimensionality" more than the Euclidean distance.

4.4 Final Conclusion

The experiments conducted lead to three major conclusions:

1. **Data Volume Matters:** Increasing the training size from 20,000 to 60,000 provided a steady 2-3% boost in accuracy across the board.

2. **Indexing is Neutral:** The Fashion-MNIST dataset is well-shuffled; Sequential vs. Random indexing yields negligible differences, proving the data is unbiased.

3. **Metric Superiority:** The Manhattan distance is objectively superior to the Euclidean distance for this high-dimensional task, providing a more robust neighborhood search.

5. Appendix Review:-

The complete numerical datasets and performance metrics for all experimental trials (Questions 1 through 3) are provided in the following Appendix for further technical review.

https://github.com/esingh16/DATA604-Midterm-1

6. References:-

1. Zalando Research. *Fashion-MNIST Dataset.* Kaggle: Available at: https://www.kaggle.com/datasets/zalando-research/fashionmnist

2. Analytics Vidhya : *https://www.analyticsvidhya.com/blog/2018/03/introduction-k-neighbours-algorithm-clustering/*

3. Amazon Web Services (AWS) : *https://docs.aws.amazon.com/sagemaker/latest/dg/k-nearest-neighbors.html*

#Question 1:- Empirical Analysis of Hyperparameters N and k

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import NearestNeighbors
from scipy.stats import mode
import time

# --- 1. Load and Preprocess ---
print("Loading Fashion-MNIST...")
(train_X, train_y), (test_X, test_y) = fashion_mnist.load_data()

# Flatten 28x28 images to 784 vectors and combine everything into one pool
# This lets us sample exactly N per class from the full 70,000 images
X_all = np.concatenate([train_X, test_X]).reshape(-1, 784).astype('float32')
         / 255.0
y_all = np.concatenate([train_y, test_y])

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# --- 2. Setup Hyperparameters ---
N_values = range(1000, 6500, 500) # 1000, 1500... 6000
k_list = [1, 2, 5, 10, 15, 20]
results = []

# --- 3. The Main Experiment Loop ---
# We loop through N first to optimize the distance calculations
for N in N_values:
    print(f"\nRunning experiment for N={N} (Total Train: {N*10})")
    start_n = time.time()

    # Stratified Sampling: Grab exactly N indices for each class
    train_idx = []
    test_idx = []

    for i in range(10):
        full_class_indices = np.where(y_all == i)[0]
        # Shuffle with a fixed seed so it's reproducible but fair
        np.random.seed(42)
        np.random.shuffle(full_class_indices)

        train_idx.extend(full_class_indices[:N])
        test_idx.extend(full_class_indices[N:])

    X_train, y_train = X_all[train_idx], y_all[train_idx]
```

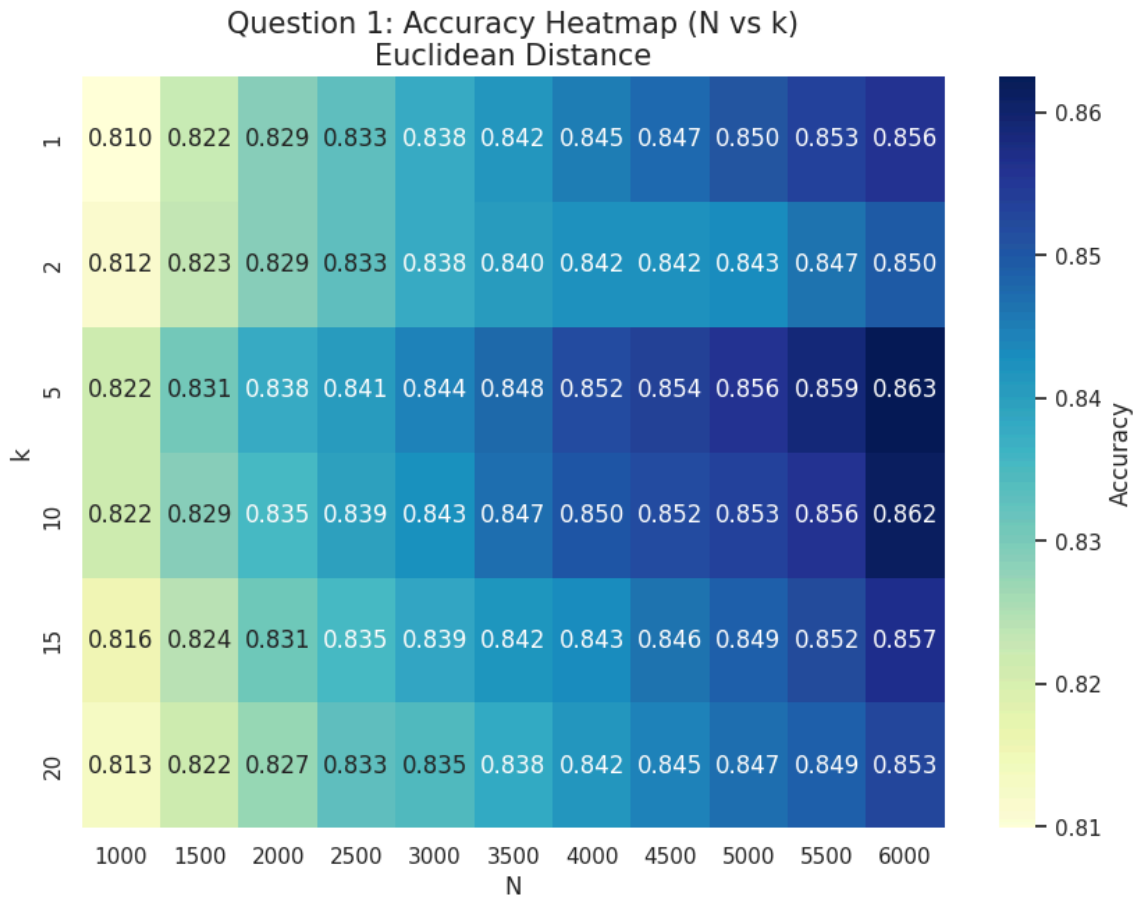
```
    X_test, y_test = X_all[test_idx], y_all[test_idx]

    # --- THE PRO OPTIMIZATION: Compute 20-NN once ---
    # We find the 20 nearest neighbors for the whole test set at once.
    # Any k < 20 is just a subset of this search.
    nn = NearestNeighbors(n_neighbors=20, metric='euclidean', n_jobs=-1)
    nn.fit(X_train)
    distances, indices = nn.kneighbors(X_test)

    # Now just loop through our k values and slice the results we already
         have
    for k in k_list:
        # Get the labels of the top k neighbors
        neighbor_labels = y_train[indices[:, :k]]

        # Majority Vote
        # Note: If there's a tie, 'mode' takes the smallest label
        predictions, _ = mode(neighbor_labels, axis=1)
        predictions = predictions.ravel()

        # Calculate Global Accuracy
        global_acc = np.mean(predictions == y_test)

        # Calculate Local Accuracies (Per Class)
        row = {'N': N, 'k': k, 'Global_Accuracy': global_acc}
        for i in range(10):
            class_mask = (y_test == i)
            local_acc = np.mean(predictions[class_mask] ==
         y_test[class_mask])
            row[f'Class_{i}_Acc'] = local_acc

        results.append(row)
        print(f"  k={k:2} | Global Acc: {global_acc:.4f}")

    print(f"  N={N} took {time.time() - start_n:.2f}s")

# --- 4. Save Results ---
df = pd.DataFrame(results)
df.to_csv('Question1_Results_Optimized.csv', index=False)

# --- 5. High-Quality Representations (Visuals) ---
sns.set_theme(style="whitegrid")

# Plot A: The Heatmap (Shows the "Sweet Spot")
plt.figure(figsize=(10, 7))
pivot = df.pivot(index="k", columns="N", values="Global_Accuracy")
sns.heatmap(pivot, annot=True, fmt=".3f", cmap="YlGnBu", cbar_kws={'label':
         'Accuracy'})
plt.title("Question 1: Accuracy Heatmap (N vs k)\nEuclidean Distance",
         fontsize=15)
plt.savefig('Q1_Heatmap.png', dpi=300)

# Plot B: Accuracy Trends
```

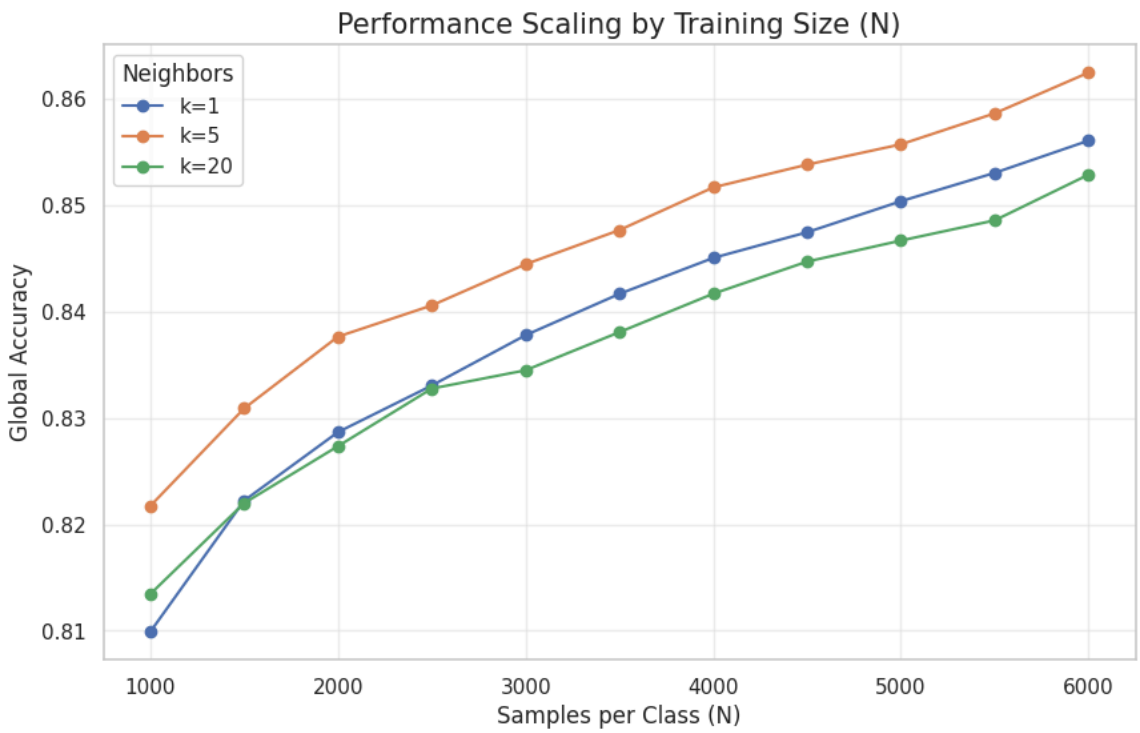
```python
plt.figure(figsize=(10, 6))
for k in [1, 5, 20]:
    subset = df[df['k'] == k]
    plt.plot(subset['N'], subset['Global_Accuracy'], marker='o', label=f'k=
         {k}')

plt.title("Performance Scaling by Training Size (N)", fontsize=15)
plt.xlabel("Samples per Class (N)")
plt.ylabel("Global Accuracy")
plt.legend(title="Neighbors")
plt.grid(True, alpha=0.3)
plt.savefig('Q1_Trends.png', dpi=300)

# Plot C: Class Difficulty (at the Best N/k)
best_row = df.loc[df['Global_Accuracy'].idxmax()]
class_accs = [best_row[f'Class_{i}_Acc'] for i in range(10)]

plt.figure(figsize=(12, 5))
colors = sns.color_palette("viridis", 10)
bars = plt.bar(class_names, class_accs, color=colors)
plt.axhline(best_row['Global_Accuracy'], color='red', linestyle='--',
         label='Global Avg')
plt.title(f"Class Stability at Optimal Config (N={int(best_row['N'])}, k=
         {int(best_row['k'])})", fontsize=15)
plt.ylabel("Accuracy")
plt.xticks(rotation=30)
plt.legend()
plt.savefig('Q1_Class_Stability.png', dpi=300)

print("\nDone! CSV and 3 high-res plots generated.")
```

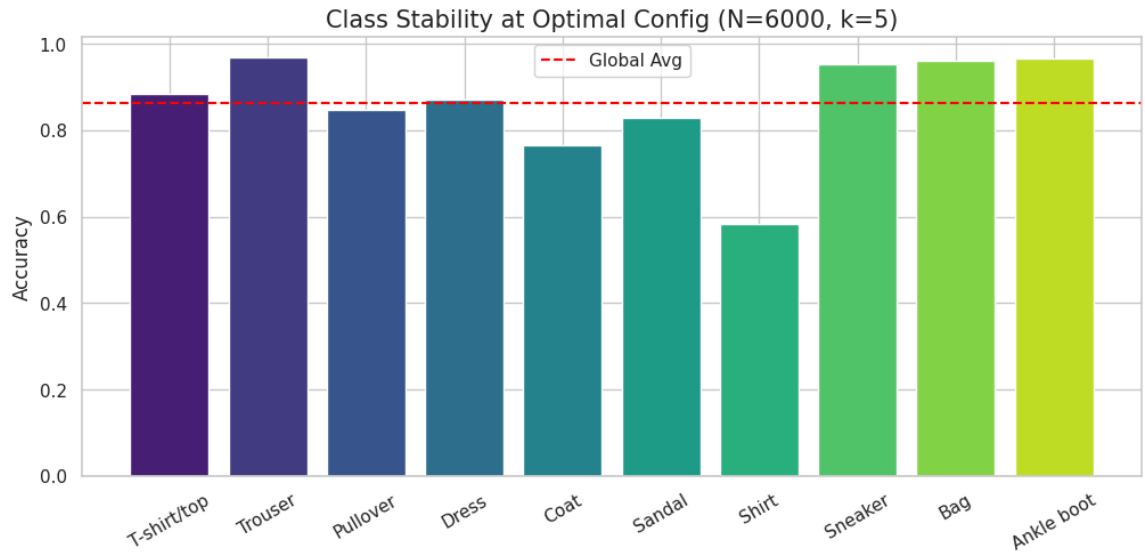
```
Loading Fashion-MNIST...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Running experiment for N=1000 (Total Train: 10000)
  k= 1 | Global Acc: 0.8099
  k= 2 | Global Acc: 0.8119
  k= 5 | Global Acc: 0.8217
  k=10 | Global Acc: 0.8219
  k=15 | Global Acc: 0.8155
  k=20 | Global Acc: 0.8135
  N=1000 took 34.71s
```

```
Running experiment for N=1500 (Total Train: 15000)
  k= 1 | Global Acc: 0.8222
  k= 2 | Global Acc: 0.8231
  k= 5 | Global Acc: 0.8309
  k=10 | Global Acc: 0.8287
  k=15 | Global Acc: 0.8241
  k=20 | Global Acc: 0.8220
  N=1500 took 39.86s

Running experiment for N=2000 (Total Train: 20000)
  k= 1 | Global Acc: 0.8287
  k= 2 | Global Acc: 0.8288
  k= 5 | Global Acc: 0.8377
  k=10 | Global Acc: 0.8353
  k=15 | Global Acc: 0.8311
  k=20 | Global Acc: 0.8274
  N=2000 took 59.81s

Running experiment for N=2500 (Total Train: 25000)
  k= 1 | Global Acc: 0.8331
  k= 2 | Global Acc: 0.8330
  k= 5 | Global Acc: 0.8406
  k=10 | Global Acc: 0.8394
  k=15 | Global Acc: 0.8354
  k=20 | Global Acc: 0.8328
  N=2500 took 54.88s

Running experiment for N=3000 (Total Train: 30000)
  k= 1 | Global Acc: 0.8378
  k= 2 | Global Acc: 0.8376
  k= 5 | Global Acc: 0.8445
  k=10 | Global Acc: 0.8427
  k=15 | Global Acc: 0.8385
  k=20 | Global Acc: 0.8345
  N=3000 took 58.10s

Running experiment for N=3500 (Total Train: 35000)
  k= 1 | Global Acc: 0.8417
  k= 2 | Global Acc: 0.8405
  k= 5 | Global Acc: 0.8477
  k=10 | Global Acc: 0.8470
  k=15 | Global Acc: 0.8419
  k=20 | Global Acc: 0.8381
  N=3500 took 58.52s

Running experiment for N=4000 (Total Train: 40000)
  k= 1 | Global Acc: 0.8451
  k= 2 | Global Acc: 0.8423
  k= 5 | Global Acc: 0.8517
  k=10 | Global Acc: 0.8501
  k=15 | Global Acc: 0.8432
  k=20 | Global Acc: 0.8417
  N=4000 took 58.15s
```

```
Running experiment for N=4500 (Total Train: 45000)
  k= 1 | Global Acc: 0.8475
  k= 2 | Global Acc: 0.8420
  k= 5 | Global Acc: 0.8538
  k=10 | Global Acc: 0.8519
  k=15 | Global Acc: 0.8464
  k=20 | Global Acc: 0.8447
  N=4500 took 55.72s

Running experiment for N=5000 (Total Train: 50000)
  k= 1 | Global Acc: 0.8504
  k= 2 | Global Acc: 0.8428
  k= 5 | Global Acc: 0.8558
  k=10 | Global Acc: 0.8532
  k=15 | Global Acc: 0.8488
  k=20 | Global Acc: 0.8467
  N=5000 took 49.30s

Running experiment for N=5500 (Total Train: 55000)
  k= 1 | Global Acc: 0.8531
  k= 2 | Global Acc: 0.8465
  k= 5 | Global Acc: 0.8587
  k=10 | Global Acc: 0.8560
  k=15 | Global Acc: 0.8521
  k=20 | Global Acc: 0.8486
  N=5500 took 38.01s

Running experiment for N=6000 (Total Train: 60000)
  k= 1 | Global Acc: 0.8561
  k= 2 | Global Acc: 0.8496
  k= 5 | Global Acc: 0.8625
  k=10 | Global Acc: 0.8615
  k=15 | Global Acc: 0.8566
  k=20 | Global Acc: 0.8529
  N=6000 took 27.24s

Done! CSV and 3 high-res plots generated.
```


Question 1: Accuracy Heatmap (N vs k)
Euclidean Distance
k
1 0.810 0.822 0.829 0.833 0.838 0.842 0.845 0.847 0.850 0.853 0.856
2 0.812 0.823 0.829 0.833 0.838 0.840 0.842 0.842 0.843 0.847 0.850
5 0.822 0.831 0.838 0.841 0.844 0.848 0.852 0.854 0.856 0.859 0.863
10 0.822 0.829 0.835 0.839 0.843 0.847 0.850 0.852 0.853 0.856 0.862
15 0.816 0.824 0.831 0.835 0.839 0.842 0.843 0.846 0.849 0.852 0.857
20 0.813 0.822 0.827 0.833 0.835 0.838 0.842 0.845 0.847 0.849 0.853
1000 1500 2000 2500 3000 3500 4000 4500 5000 5500 6000
N
Accuracy
0.86
0.85
0.84
0.83
0.82
0.81


Performance Scaling by Training Size (N)
Neighbors
k=1
k=5
k=20
Global Accuracy
0.86
0.85
0.84
0.83
0.82
0.81
1000
2000
3000
4000
5000
6000
Samples per Class (N)



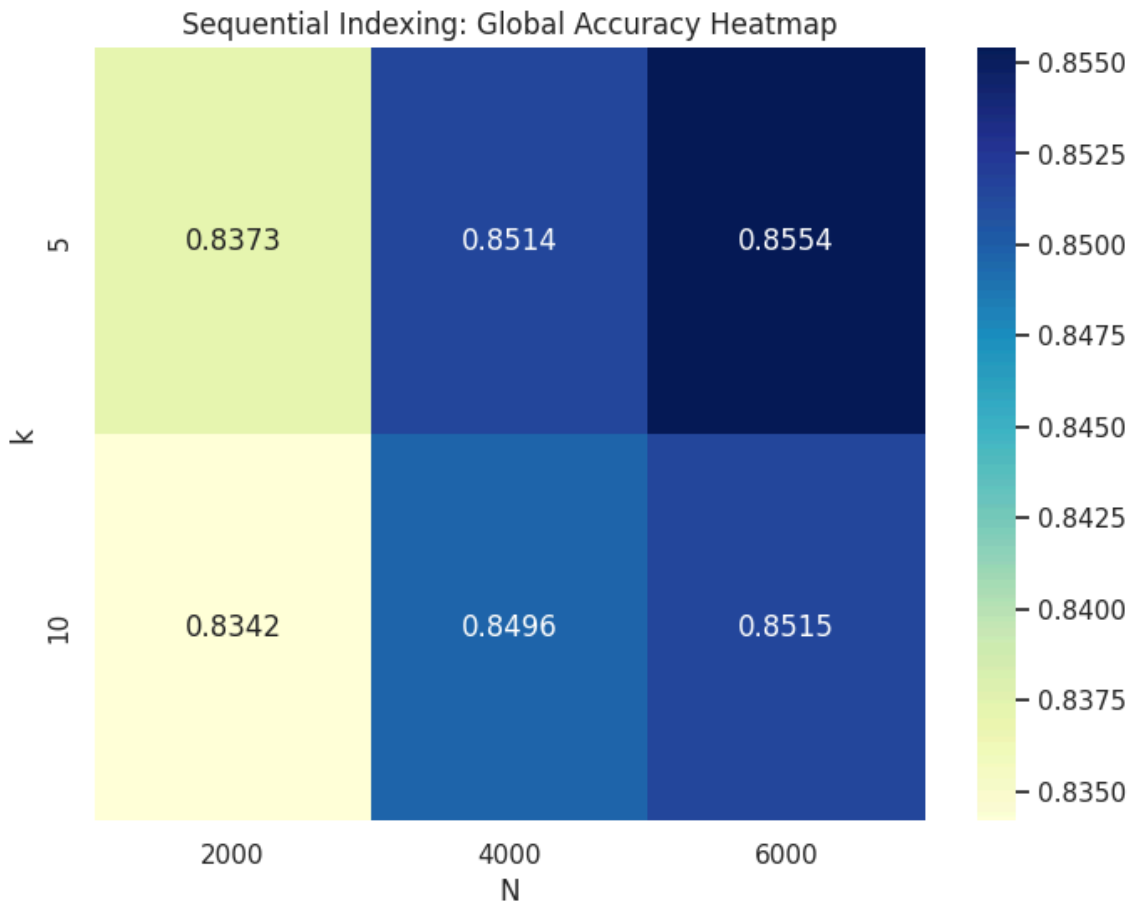
#Question 2:- Influence of Indexing Methods on Model Bias

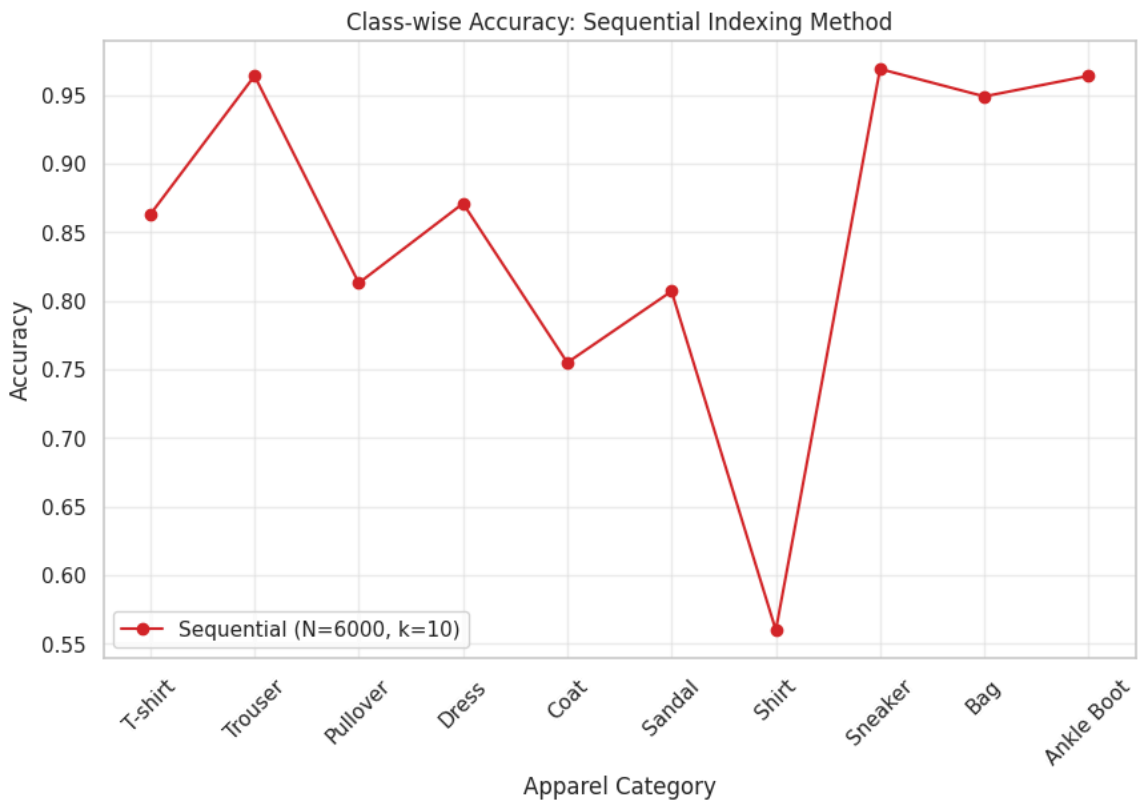
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import NearestNeighbors
from scipy.stats import mode

# --- 1. Load Data ---
print("Initializing Data for Indexing Comparison...")
(train_X, train_y), (test_X, test_y) = fashion_mnist.load_data()
X_all = np.concatenate([train_X, test_X]).reshape(-1, 784).astype('float32')
         / 255.0
y_all = np.concatenate([train_y, test_y])
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal',
         'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']

# Parameters for Q2
N_values = [2000, 4000, 6000]
k_values = [5, 10]
methods = ['Random', 'Sequential']
results_q2 = []

# --- 2. Execution Loop ---
for method in methods:
    print(f"\n--- Testing Indexing Method: {method} ---")
    for N in N_values:
        total_samples = N * 10
        print(f"Loading dataset: {total_samples} training samples...")

        train_idx, test_idx = [], []
        for i in range(10):
            full_class_indices = np.where(y_all == i)[0]
            if method == 'Random':
```

```
                np.random.seed(42)
                idx = np.random.choice(full_class_indices, N, replace=False)
            else:
                idx = full_class_indices[:N] # Sequential: first N images

            train_idx.extend(idx)
            remaining = np.setdiff1d(full_class_indices, idx)
            test_idx.extend(remaining[:1000])

        X_train, y_train = X_all[train_idx], y_all[train_idx]
        X_test, y_test = X_all[test_idx], y_all[test_idx]

        # THE FIX: Added algorithm='brute' for consistency and stability
        nn = NearestNeighbors(n_neighbors=10, metric='euclidean',
         algorithm='brute', n_jobs=-1)
        nn.fit(X_train)
        distances, indices = nn.kneighbors(X_test)

        for k in k_values:
            neighbor_labels = y_train[indices[:, :k]]
            predictions, _ = mode(neighbor_labels, axis=1, keepdims=True)
            predictions = predictions.ravel()

            acc = np.mean(predictions == y_test)
            res = {'Method': method, 'N': N, 'k': k, 'Global_Accuracy': acc}
            for i in range(10):
                res[f'Class_{i}_Acc'] = np.mean(predictions[y_test == i] ==
         i)

            results_q2.append(res)
            print(f"  Result: {method} | N={N}, k={k} | Accuracy: {acc:.4f}")

df_q2 = pd.DataFrame(results_q2)
df_q2.to_csv('Question2_Results.csv', index=False)

# --- 3. Visualizations (Matching Style) ---

# Heatmap for Sequential Method
plt.figure(figsize=(8, 6))
pivot_df = df_q2[df_q2['Method'] == 'Sequential'].pivot(index='k',
         columns='N', values='Global_Accuracy')
sns.heatmap(pivot_df, annot=True, fmt=".4f", cmap="YlGnBu")
plt.title("Sequential Indexing: Global Accuracy Heatmap")
plt.savefig('Q2_Sequential_Heatmap.png', dpi=300)

# Class-wise Accuracy for Sequential (Best Config)
plt.figure(figsize=(10, 6))
best_seq = df_q2[(df_q2['Method'] == 'Sequential') & (df_q2['N'] == 6000) &
         (df_q2['k'] == 10)]
class_accs = [best_seq[f'Class_{i}_Acc'].values[0] for i in range(10)]

plt.plot(class_names, class_accs, marker='o', linestyle='-', color='tab:red',
         label='Sequential (N=6000, k=10)')
```

```python
plt.title("Class-wise Accuracy: Sequential Indexing Method")
plt.xlabel("Apparel Category")
plt.ylabel("Accuracy")
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.legend()
plt.savefig('Q2_Sequential_Stability.png', dpi=300)

print("\nProcessing Complete. Question 2 images saved.")
```

```
Initializing Data for Indexing Comparison...

--- Testing Indexing Method: Random ---
Loading dataset: 20000 training samples...
  Result: Random | N=2000, k=5 | Accuracy: 0.8375
  Result: Random | N=2000, k=10 | Accuracy: 0.8348
Loading dataset: 40000 training samples...
  Result: Random | N=4000, k=5 | Accuracy: 0.8500
  Result: Random | N=4000, k=10 | Accuracy: 0.8511
Loading dataset: 60000 training samples...
  Result: Random | N=6000, k=5 | Accuracy: 0.8625
  Result: Random | N=6000, k=10 | Accuracy: 0.8615

--- Testing Indexing Method: Sequential ---
Loading dataset: 20000 training samples...
  Result: Sequential | N=2000, k=5 | Accuracy: 0.8373
  Result: Sequential | N=2000, k=10 | Accuracy: 0.8342
Loading dataset: 40000 training samples...
  Result: Sequential | N=4000, k=5 | Accuracy: 0.8514
  Result: Sequential | N=4000, k=10 | Accuracy: 0.8496
Loading dataset: 60000 training samples...
  Result: Sequential | N=6000, k=5 | Accuracy: 0.8554
  Result: Sequential | N=6000, k=10 | Accuracy: 0.8515

Processing Complete. Question 2 images saved.
```





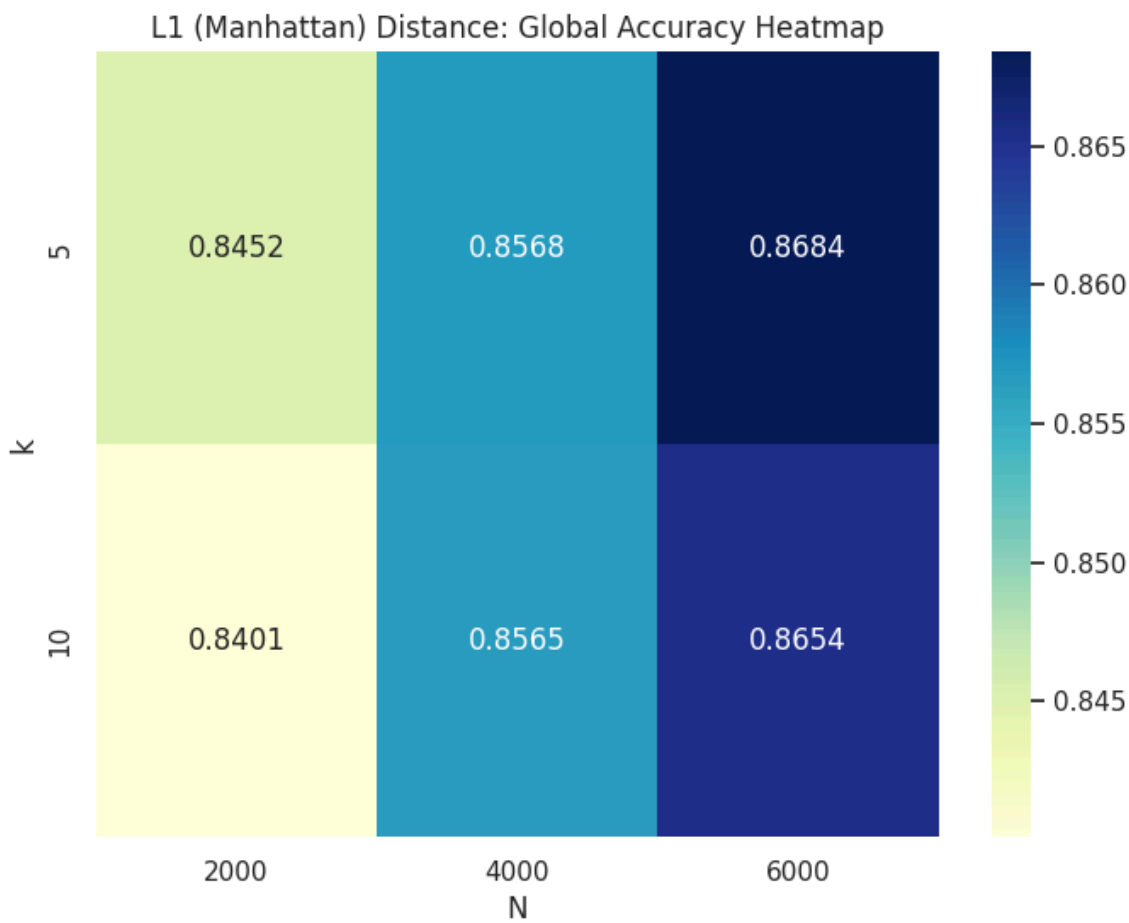
#Question 3:- Impact of Distance Metrics on Image Classification

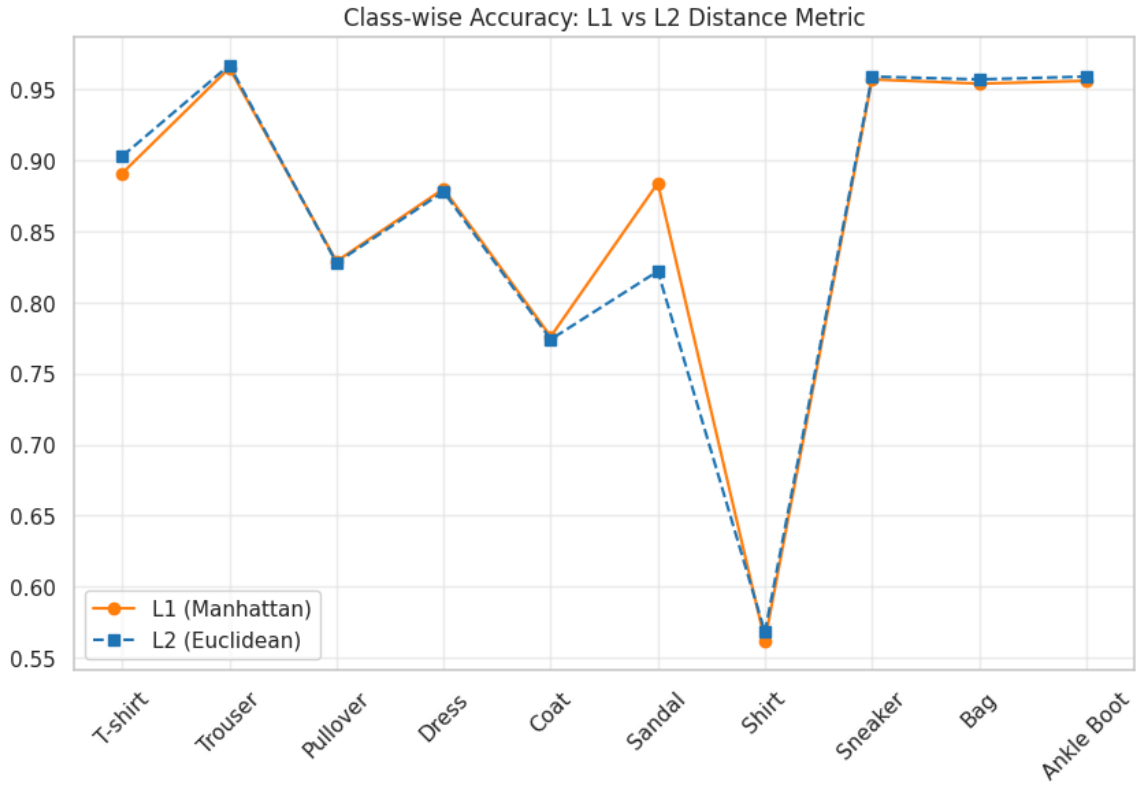
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import NearestNeighbors
from scipy.stats import mode

# --- 1. Load Data ---
print("Initializing Metric Comparison Experiment (L1 vs L2)...")
(train_X, train_y), (test_X, test_y) = fashion_mnist.load_data()
X_all = np.concatenate([train_X, test_X]).reshape(-1, 784).astype('float32')
         / 255.0
y_all = np.concatenate([train_y, test_y])
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal',
         'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']

N_values = [2000, 4000, 6000]
k_values = [5, 10]
metrics = ['manhattan', 'euclidean']
results_q3 = []

# --- 2. Execution Loop ---
for met in metrics:
    print(f"\n--- Testing Distance Metric: {met.upper()} ---")
    for N in N_values:
        print(f"Loading dataset: {N*10} training samples...")

        train_idx, test_idx = [], []
        for i in range(10):
            full_class_indices = np.where(y_all == i)[0]
            np.random.seed(42)
            idx = np.random.choice(full_class_indices, N, replace=False)
            train_idx.extend(idx)
            remaining = np.setdiff1d(full_class_indices, idx)
            test_idx.extend(remaining[:1000])

        X_train, y_train = X_all[train_idx], y_all[train_idx]
        X_test, y_test = X_all[test_idx], y_all[test_idx]

        # THE FIX: Added algorithm='brute' to handle Manhattan metric
         properly
        nn = NearestNeighbors(n_neighbors=10, metric=met, algorithm='brute',
         n_jobs=-1)
        nn.fit(X_train)
        distances, indices = nn.kneighbors(X_test)

        for k in k_values:
            neighbor_labels = y_train[indices[:, :k]]
            predictions, _ = mode(neighbor_labels, axis=1, keepdims=True)
            predictions = predictions.ravel()
```

```
            acc = np.mean(predictions == y_test)
            res = {'Metric': met, 'N': N, 'k': k, 'Global_Accuracy': acc}
            for i in range(10):
                res[f'Class_{i}_Acc'] = np.mean(predictions[y_test == i] ==
        i)

            results_q3.append(res)
            print(f"  Result: {met} | N={N}, k={k} | Accuracy: {acc:.4f}")

df_q3 = pd.DataFrame(results_q3)
df_q3.to_csv('Question3_Results.csv', index=False)

# --- 3. Visualizations ---
# Heatmap for Manhattan
plt.figure(figsize=(8, 6))
l1_data = df_q3[df_q3['Metric'] == 'manhattan'].pivot(index='k', columns='N',
        values='Global_Accuracy')
sns.heatmap(l1_data, annot=True, fmt=".4f", cmap="YlGnBu")
plt.title("L1 (Manhattan) Distance: Global Accuracy Heatmap")
plt.savefig('Q3_L1_Heatmap.png', dpi=300)

# Class-wise Metric Comparison
plt.figure(figsize=(10, 6))
best_l1 = df_q3[(df_q3['Metric'] == 'manhattan') & (df_q3['N'] == 6000) &
        (df_q3['k'] == 10)]
best_l2 = df_q3[(df_q3['Metric'] == 'euclidean') & (df_q3['N'] == 6000) &
        (df_q3['k'] == 10)]

l1_accs = [best_l1[f'Class_{i}_Acc'].values[0] for i in range(10)]
l2_accs = [best_l2[f'Class_{i}_Acc'].values[0] for i in range(10)]

plt.plot(class_names, l1_accs, marker='o', label='L1 (Manhattan)',
        color='tab:orange')
plt.plot(class_names, l2_accs, marker='s', label='L2 (Euclidean)',
        color='tab:blue', linestyle='--')
plt.title("Class-wise Accuracy: L1 vs L2 Distance Metric")
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('Q3_Metric_Stability_Comparison.png', dpi=300)

plt.show()
```

```
Initializing Metric Comparison Experiment (L1 vs L2)...

--- Testing Distance Metric: MANHATTAN ---
Loading dataset: 20000 training samples...
  Result: manhattan | N=2000, k=5 | Accuracy: 0.8452
  Result: manhattan | N=2000, k=10 | Accuracy: 0.8401
Loading dataset: 40000 training samples...
  Result: manhattan | N=4000, k=5 | Accuracy: 0.8568
  Result: manhattan | N=4000, k=10 | Accuracy: 0.8565
Loading dataset: 60000 training samples...
  Result: manhattan | N=6000, k=5 | Accuracy: 0.8684
```

```
  Result: manhattan | N=6000, k=10 | Accuracy: 0.8654

--- Testing Distance Metric: EUCLIDEAN ---
Loading dataset: 20000 training samples...
  Result: euclidean | N=2000, k=5 | Accuracy: 0.8375
  Result: euclidean | N=2000, k=10 | Accuracy: 0.8348
Loading dataset: 40000 training samples...
  Result: euclidean | N=4000, k=5 | Accuracy: 0.8500
  Result: euclidean | N=4000, k=10 | Accuracy: 0.8511
Loading dataset: 60000 training samples...
  Result: euclidean | N=6000, k=5 | Accuracy: 0.8625
  Result: euclidean | N=6000, k=10 | Accuracy: 0.8615
```




Class-wise Accuracy: L1 vs L2 Distance Metric
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0.55
L1 (Manhattan)
L2 (Euclidean)
T-shirt
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle Boot

!!!!!Thank You!!!!!